

5/8/2026

E-Ink project

Microprocessor Based Systems

Under the supervision of:

Dr. Medhat Hussien

Eng, Shady Medhat

By:

Amr Hassan

22/07162

Batool Sherif

22/07271

Moataz Hazem

19/02383

Contents

Project Overview:	1
System Architecture:	1
mobile app:	1
Communication layer	1
Embedded layer	1
Mobile Application:	2
Bluetooth Connectivity:	2
Payload Construction:	2
User Interface:	3
Firmware:	4
Clock Source:	4
Display Driver:	4
UART and CSV Parsing:	6
Button Handling:	6
Battery Monitoring:	7
Timer:	7
Pin table and firmware structure:	7
Wiring diagram:	8
Data Flow Summary:	9
Further improvements:	9
Conclusion:	9

Tables:

Table 1 Pin Table	7
Table 2 Microcontroller Comparison	9

Figures:

Figure 1 High-level project structure	2
Figure 2 Mobile App	3
Figure 3 Column scan rendering architecture	4
Figure 4 E-Ink layout	5
Figure 5 Button handling	6
Figure 6 firmware internal module structure	8
Figure 7 wiring diagram.....	8

Project Overview:

This project is an embedded smart display system designed to help a student manage daily tasks, upcoming exams, and habit tracking. It consists of two tightly coupled components: a React Native mobile application running on Android, and an ATmega32-based hardware device driving a 250×122 e-paper display. The user enters data through the app and sends it wirelessly to the microcontroller over Bluetooth Classic via an HC-06 module. The ATmega32 parses the received data, renders it onto the e-paper display, and exposes two hardware navigation buttons for scrolling through tasks and logging habit progress.

System Architecture:

The system is divided into three layers.

mobile app: A React Native application provides the user interface for data entry. The user fills in up to ten tasks with associated times, an upcoming exam with its date, and a habit to track. A live payload preview shows the exact CSV string that will be transmitted before the user sends it. The app manages Bluetooth connectivity, displaying paired devices from Android's bonded-device list and connecting over RFCOMM.

Communication layer (HC-06 + UART): The HC-06 Bluetooth Classic module acts as a transparent serial bridge. It receives the UTF-8 CSV string from the phone and forwards it byte by byte to the ATmega32's UART receive pin at 9600 baud. No custom protocol framing is needed — the payload is terminated with a newline character (`\n`), which the microcontroller uses as the end-of-message.

Embedded layer (ATmega32 + e-paper): The microcontroller runs entirely from its internal 1 MHz oscillator — no external crystal is used. It initializes SPI to drive the e-paper display, sets up Timer0 in CTC mode to generate a 1ms system tick, configures INT0 and INT1 as falling-edge external interrupts for the two push buttons, enables the UART receive interrupt, and starts the ADC for battery voltage measurement. The main loop continuously polls for completed UART lines and release button events, updating the display as needed.

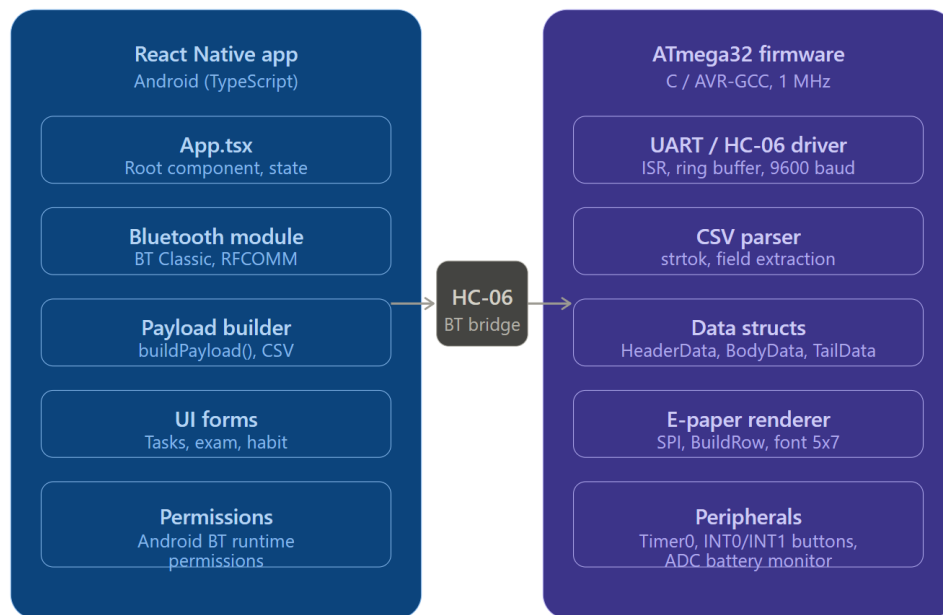


Figure 1 High-level project structure

Mobile Application:

Bluetooth Connectivity:

The app uses the react native Bluetooth Classic library to communicate with the HC-06 module. Because HC-06 is a Bluetooth Classic (SPP profile) device, it must first be paired through Android's system Settings before the app can connect to it. The app calls `getBondedDevices()` to retrieve all already-paired devices and presents them in a bottom-sheet modal. Selecting a device calls `connectToDevice(address)`, which opens an RFCOMM socket. Disconnecting calls `disconnectFromDevice(address)`.

On Android 12 and above, the app requests `BLUETOOTH_SCAN` and `BLUETOOTH_CONNECT` runtime permissions before attempting to list devices. On Android 6–11, it requests `ACCESS_FINE_LOCATION`, which is required for Bluetooth device discovery on those API levels.

Payload Construction:

The `buildPayload` function assembles all form fields into a single comma-separated string terminated with a newline. The format is:

```
<date>,<day>,<task1>,<time1>,...,<task_n>,<time_n>,<exam name>,<exam date>,<habit name>\n
```

The date is formatted as D MMM (e.g. 8 May) and the day as the full weekday name (e.g. Friday). Task and time fields are sent in interleaved pairs. The exam and habit fields occupy the final three positions. This ordering is matched exactly by the parsing logic on the ATmega32.

User Interface:

The interface presents four card sections — Tasks, Exam, Habit, and a Payload Preview — inside a scroll view with keyboard-avoiding behavior. The connection status bar at the top displays an animated pulsing ring when connected and shows the device name. The send button is visually dimmed when no device is connected. The dark-themed color scheme uses a navy background with high-contrast accents.

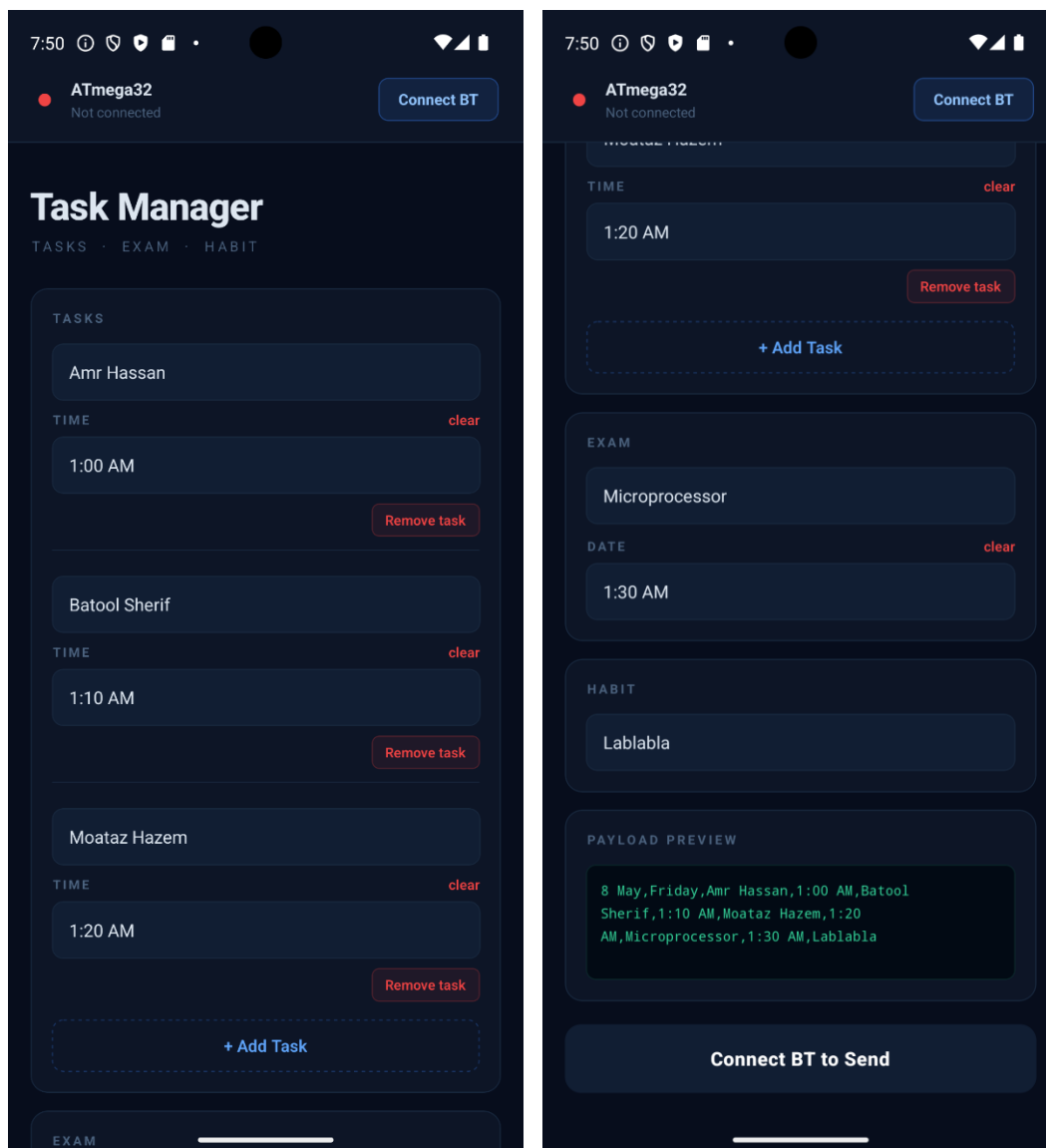


Figure 2 Mobile App

Firmware:

Clock Source:

The ATmega32 runs entirely from its internal RC oscillator at 1 MHz, the factory default fuse setting. No external crystal or resonator is used. F_CPU is defined as 1000000UL throughout the firmware, which correctly calibrates all timing-dependent functions: _delay_ms(), the Timer0 CTC compare value, and the UART baud rate register. At 1 MHz and 9600 baud, the UART baud rate register resolves to UBRR = $(1,000,000 / (16 \times 9600)) - 1 = 5$.

Display Driver:

The E-paper module is a 2.13-inch 250×122 monochrome panel. The firmware communicates with the display over the ATmega32 hardware SPI peripheral using four additional control lines: Chip Select, Data/Command, Reset, and Busy. Display initialization follows the standard sequence of software reset, driver output configuration, data entry mode selection, RAM address window setup, and RAM counter initialization.

To minimize SRAM usage, the renderer uses a streamed column-scan architecture rather than a full framebuffer. Instead of allocating a complete 250×122 bitmap (which would require approximately 4 KB of RAM and exceed the ATmega32's 2 KB SRAM limit), the firmware maintains a temporary 16-byte column buffer. Each rendering pass generates one logical display column at a time through the BuildRow() function and immediately transmits it to the display over SPI.

The rendering pipeline iterates through all 250 logical columns using a rotated coordinate mapping in which the controller RAM scan order is translated into the display's landscape orientation. For each logical column, BuildRow() composes the visible content into the temporary buffer before transmission.

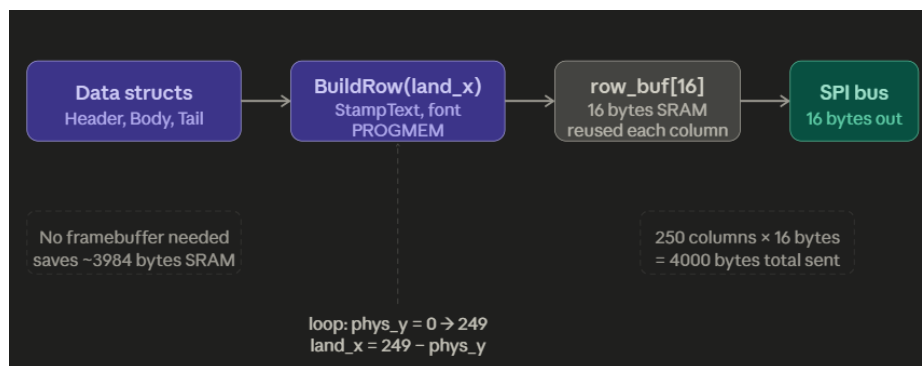


Figure 3 Column scan rendering architecture

The screen layout is divided into three logical regions:

- Header region ($y \approx 0-21$): date string at $x=4$, day string at $x=109$, and battery icon at $x \approx 220-239$.
- Body region ($y \approx 22-90$): up to four tasks from the currently selected page, each containing a task label on the left and a fixed-position time string near the right edge.
- Tail region ($y \approx 91-122$): exam name and exam date on one line, followed by a habit tracker label and circular filled/unfilled progress indicators.

Text rendering uses a 5×7 monochrome bitmap font stored in flash memory using PROGMEM in order to conserve SRAM. The StampText() function maps the current logical column to the appropriate column and clears the corresponding pixel bits within the temporary column buffer.

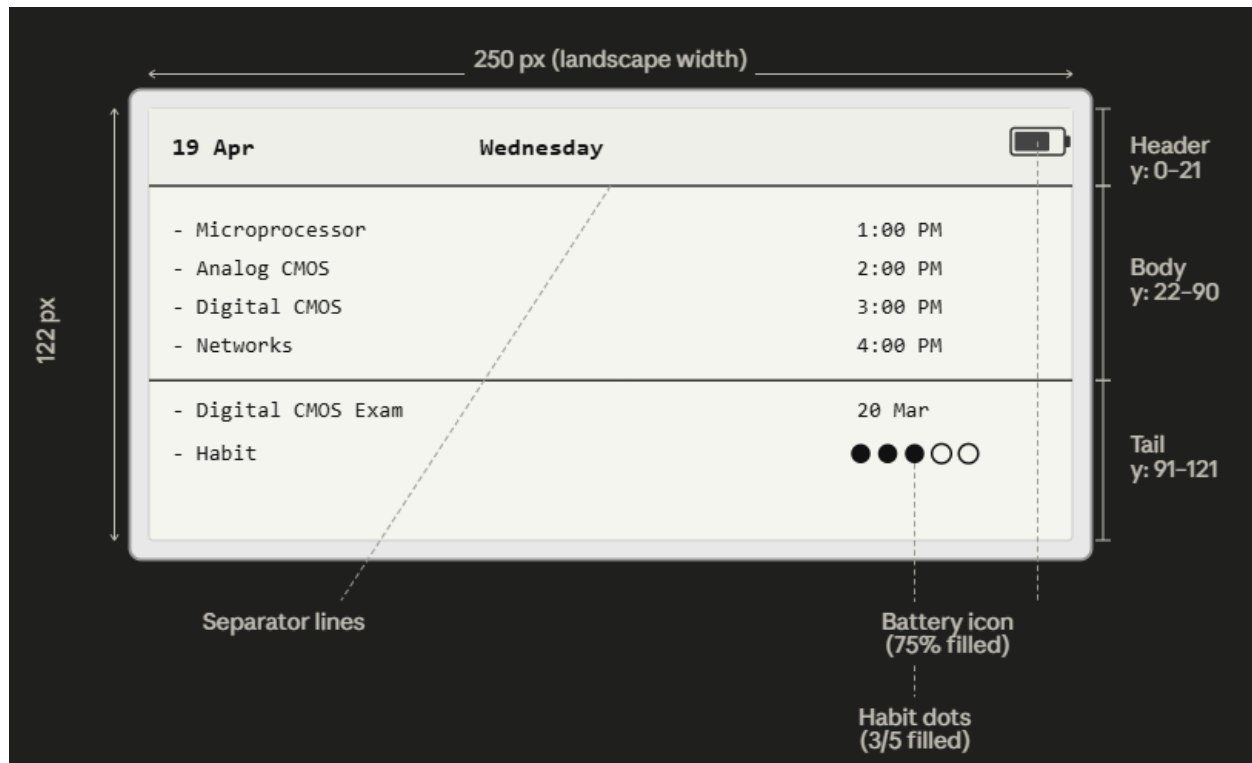


Figure 4 E-Ink layout

UART and CSV Parsing:

The UART is configured for 9600 baud. With $F_{\text{CPU}} = 1000000$, UBRR = 5 is loaded into the baud rate registers. A 256-byte circular ring buffer uses uint8_t head and tail indices, who's natural 8-bit overflow provides the correct modulo-256 wrap without any masking arithmetic. The ISR stores each receive byte and set a line_ready flag when a newline is detected.

The main loop calls Receive_Line(), which temporarily disables the receive interrupt while copying from the ring buffer to a local line buffer, flushing any trailing `\r\n` bytes, and re-enabling the interrupt. Parse_CSV() tokenizes the line using strtok() into a field array of up to 20 fields, each up to 24 characters. Extract_And_Update() then maps field indices to the HeaderData, BodyData, and TailData structs, triggers a full-screen refresh, and resets the body selection to page zero.

A minimum field count of 5 is enforced before any struct is written, preventing partial data from corrupting the display in the event of a truncated or malformed transmission.

Button Handling:

Two push buttons are connected to PD2 (INT0, Down) and PD3 (INT1, Up) with internal pull-ups enabled. Both external interrupt ISRs are triggered by falling edge. The ISRs record the press time counter but perform no delay or debouncing logic, keeping interrupt latency minimal.

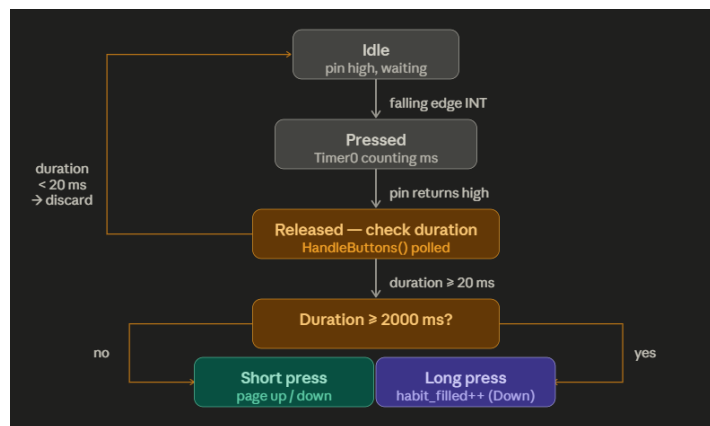


Figure 5 Button handling

Debouncing is handled in HandleButtons(), which is polled from the main loop. The press is considered valid only if the button has been released (pin reads high) and at least 20ms have elapsed since the falling edge. Pulses shorter than 20ms are discarded as contact bounce.

Long-press detection (≥ 2000 ms) is implemented on the Down button to increment the habit fill counter without navigating, allowing the user to mark a habit as done. Short-press navigates the task page down. Pressing the Up button navigates up. Long-press on the Up button is reserved for future use.

Battery Monitoring:

The firmware uses the AVR's internal bandgap reference to measure AVCC. With the ADC multiplexer set to the bandgap channel and the reference set to AVCC, $ADC = (1.1 \text{ V} \times 1023) / AVCC$, from which AVCC is derived. Five readings are averaged with 80ms spacing to filter noise. The resulting voltage is mapped to a percentage using a lookup table with 50 mV steps from 2550 mV to 3000 mV, suitable for a 3V supply.

Timer:

Timer0 is configured in CTC mode with a prescaler of 64. The compare match value is $OCR0 = (F_CPU / (64 \times 1000)) - 1$. At $F_CPU = 1000000$, this yields $OCR0 = 14$, producing a compare match interrupt every 1ms. The ISR increments the two button-press duration counters while their respective press flags are active.

Pin table and firmware structure:

Table 1 Pin Table

Component	Atmega32 pin	Function
SDA (Display)	PB5 (MOSI)	SPI Data Out
SCL (Display)	PB7 (SCK)	SPI Clock
CS (Display)	PB4	Chip Select (Active Low)
DC (Display)	PD6	Data/Command Select
RES (Display)	PD7	Reset (Active Low)
BUSY (Display)	PD5	Busy Status Input
TX (Bluetooth HC-06)	PD0 (RXD)	Receive data through HC-06
btn 0	PD2 (INT0)	Tasks navigate down/habit
btn 1	PD3 (INT1)	Tasks navigate up

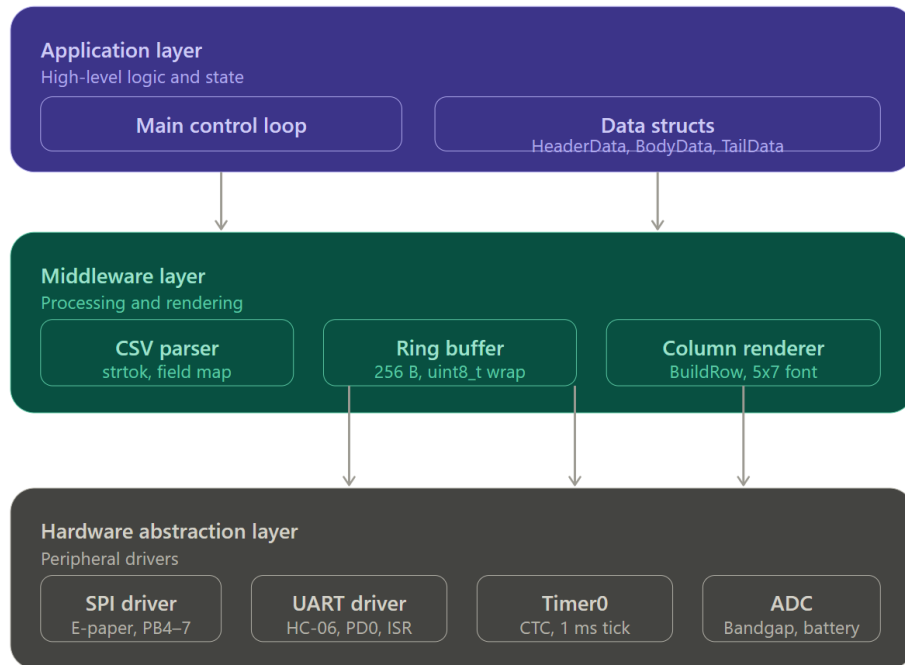


Figure 6 firmware internal module structure

Wiring diagram:

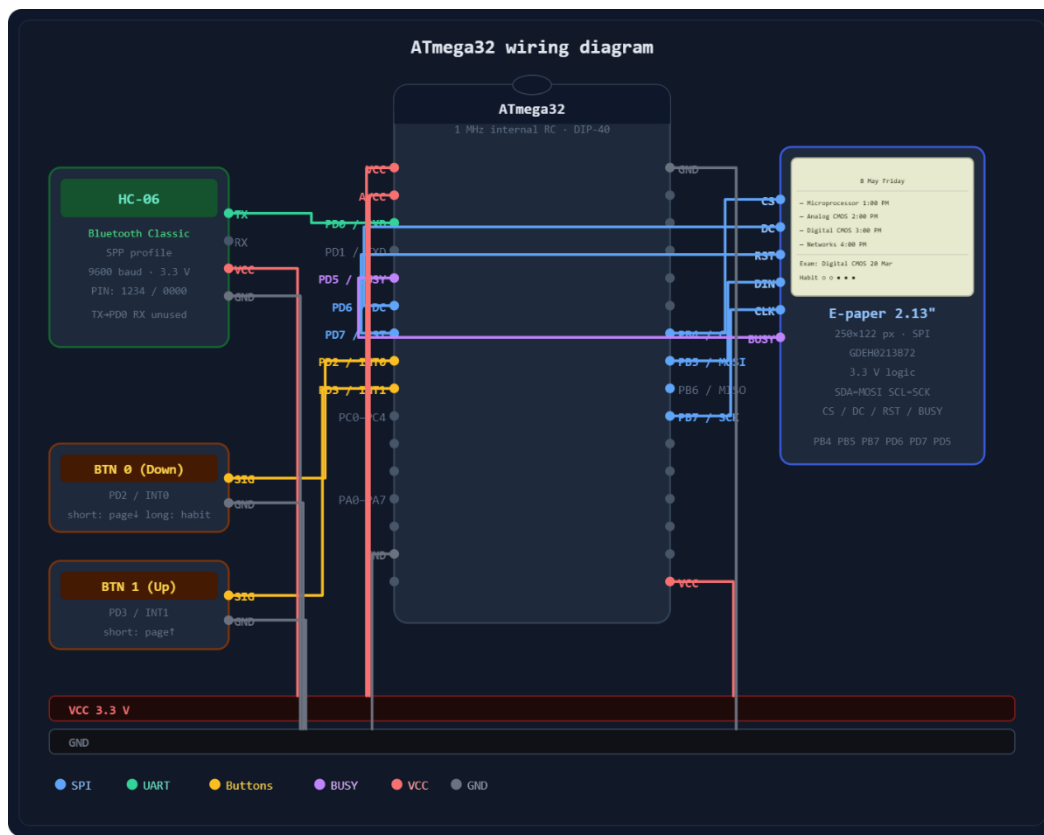


Figure 7 wiring diagram

Data Flow Summary:

1. The user fills in tasks, exam details, and a habit in the React Native app.
2. The app formats the data as a CSV string and sends it over RFCOMM to the HC-06 module.
3. HC-06 forwards the bytes to the ATmega32's UART RX pin at 9600 baud.
4. The UART ISR stores bytes in the ring buffer; the newline byte sets line_ready flag.
5. The main loop detects the line_ready flag, reads and parses the CSV, updates the data structs, and triggers a full display refresh.
6. EPD_WriteFullRAM() scans all 250 columns, calling BuildRow() for each and sending the result over SPI to the e-paper panel.
7. EPD_TriggerRefresh() sends the display update command and waits for the busy signal to clear. Then the user can navigate tasks.

Further improvements:

After analyzing the final implementation of the project, we identified significant potential for hardware optimization. While the **ATmega32A** provided a comfortable environment for development, our resource profiling indicates that the system could be migrated to a more compact and cost-effective microcontroller, such as the **ATmega8A**.

Table 2 Microcontroller Comparison

Feature	ATmega32A (Current)	ATmega8A (Proposed)	Project Requirement
Flash Memory	32 KB	8 KB	~4.5 KB (Current Code)
SRAM	2 KB	1 KB	~600 B
EEPROM	1024 Bytes	512 Bytes	Text storage
Ext. Interrupts	3	2	2
Communication	UART, SPI, I2C	UART, SPI, I2C	UART, SPI

Conclusion:

This project demonstrates a low-power student organizer using an ATmega32 and an E-Ink display, synchronized via Bluetooth. By implementing a custom column-scan rendering architecture, we successfully bypassed the SRAM limitations of the AVR platform. The final prototype serves as a comprehensive application of interrupt-driven logic and serial communication, with clear paths for further optimization on smaller microcontrollers like the ATmega8.